

16 Exemplar-based Methods

So far in this book, we have mostly focused on **parametric models**, either unconditional $p(\mathbf{y}|\boldsymbol{\theta})$ or conditional $p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is a fixed-dimensional vector of parameters. The parameters are estimated from a variable-sized dataset, $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n) : n = 1 : N\}$, but after model fitting, the data is thrown away.

In this section we consider various kinds of **nonparametric models**, that keep the training data around. Thus the effective number of parameters of the model can grow with $|\mathcal{D}|$. We focus on models that can be defined in terms of the **similarity** between a test input, $\mathbf{x}$, and each of the training inputs, $\mathbf{x}_n$. Alternatively, we can define the models in terms of a dissimilarity or distance function $d(\mathbf{x}, \mathbf{x}_n)$. Since the models keep the training examples around at test time, we call them **exemplar-based models**. (This approach is also called **instance-based learning** [AKA91], or **memory-based learning**.)

16.1 K nearest neighbor (KNN) classification

In this section, we discuss one of the simplest kind of classifier, known as the **K nearest neighbor (KNN)** classifier. The idea is as follows: to classify a new input $\mathbf{x}$, we find the K closest examples to $\mathbf{x}$ in the training set, denoted $N_K(\mathbf{x}, \mathcal{D})$, and then look at their labels, to derive a distribution over the outputs for the local region around $\mathbf{x}$. More precisely, we compute

$$p(y = c|\mathbf{x}, \mathcal{D}) = \frac{1}{K} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n = c) \quad (16.1)$$

We can then return this distribution, or the majority label.

The two main parameters in the model are the size of the neighborhood, K , and the distance metric $d(\mathbf{x}, \mathbf{x}')$. For the latter, it is common to use the **Mahalanobis distance**

$$d_{\mathbf{M}}(\mathbf{x}, \boldsymbol{\mu}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{M} (\mathbf{x} - \boldsymbol{\mu})} \quad (16.2)$$

where $\mathbf{M}$ is a positive definite matrix. If $\mathbf{M} = \mathbf{I}$, this reduces to Euclidean distance. We discuss how to learn the distance metric in Section 16.2.

Despite the simplicity of KNN classifiers, it can be shown that this approach gets within a factor of 2 of the Bayes error (which measures the performance of the best possible classifier) as $N \rightarrow \infty$ [CH67; CD14]. (Of course the convergence rate to this optimal performance may be poor in practice, for reasons we discuss in Section 16.1.2.)

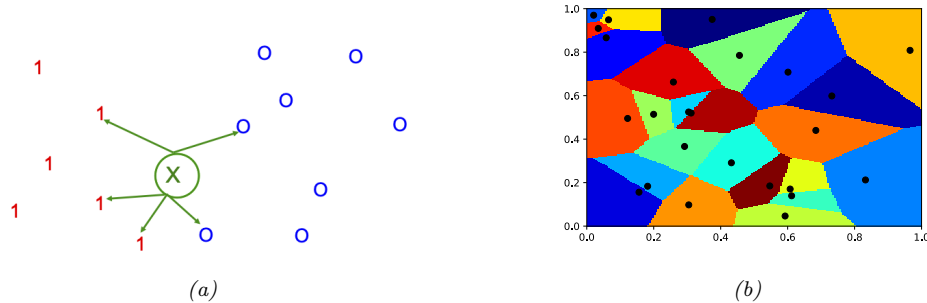


Figure 16.1: (a) Illustration of a K -nearest neighbors classifier in 2d for $K = 5$. The nearest neighbors of test point $\mathbf{x}$ have labels $\{1, 1, 1, 0, 0\}$, so we predict $p(y = 1|\mathbf{x}, \mathcal{D}) = 3/5$. (b) Illustration of the Voronoi tessellation induced by 1-NN. Adapted from Figure 4.13 of [DHS01]. Generated by `knn_voronoi_plot.ipynb`.

16.1.1 Example

We illustrate the KNN classifier in 2d in Figure 16.1(a) for $K = 5$. The test point is marked as an “x”. 3 of the 5 nearest neighbors have label 1, and 2 of the 5 have label 0. Hence we predict $p(y = 1|\mathbf{x}, \mathcal{D}) = 3/5 = 0.6$.

If we use $K = 1$, we just return the label of the nearest neighbor, so the predictive distribution becomes a delta function. A KNN classifier with $K = 1$ induces a **Voronoi tessellation** of the points (see Figure 16.1(b)). This is a partition of space which associates a region $V(\mathbf{x}_n)$ with each point $\mathbf{x}_n$ in such a way that all points in $V(\mathbf{x}_n)$ are closer to $\mathbf{x}_n$ than to any other point. Within each cell, the predicted label is the label of the corresponding training point. Thus the training error will be 0 when $K = 1$. However, such a model is usually overfitting the training set, as we show below.

Figure 16.2 gives an example of KNN applied to a 2d dataset, in which we have three classes. We see how, with $K = 1$, the method makes zero errors on the training set. As K increases, the decision boundaries become smoother (since we are averaging over larger neighborhoods), so the training error increases, as we start to underfit. This is shown in Figure 16.2(d). The test error shows the usual U-shaped curve.

16.1.2 The curse of dimensionality

The main statistical problem with KNN classifiers is that they do not work well with high dimensional inputs, due to the **curse of dimensionality**.

The basic problem is that the volume of space grows exponentially fast with dimension, so you might have to look quite far away in space to find your nearest neighbor. To make this more precise, consider this example from [HTF09, p22]. Suppose we apply a KNN classifier to data where the inputs are uniformly distributed in the D -dimensional unit cube. Suppose we estimate the density of class labels around a test point $\mathbf{x}$ by “growing” a hyper-cube around $\mathbf{x}$ until it contains a desired fraction p of the data points. The expected edge length of this cube will be $e_D(s) \triangleq p^{1/D}$; this function is plotted in Figure 16.3(b). If $D = 10$, and we want to base our estimate on 10% of the

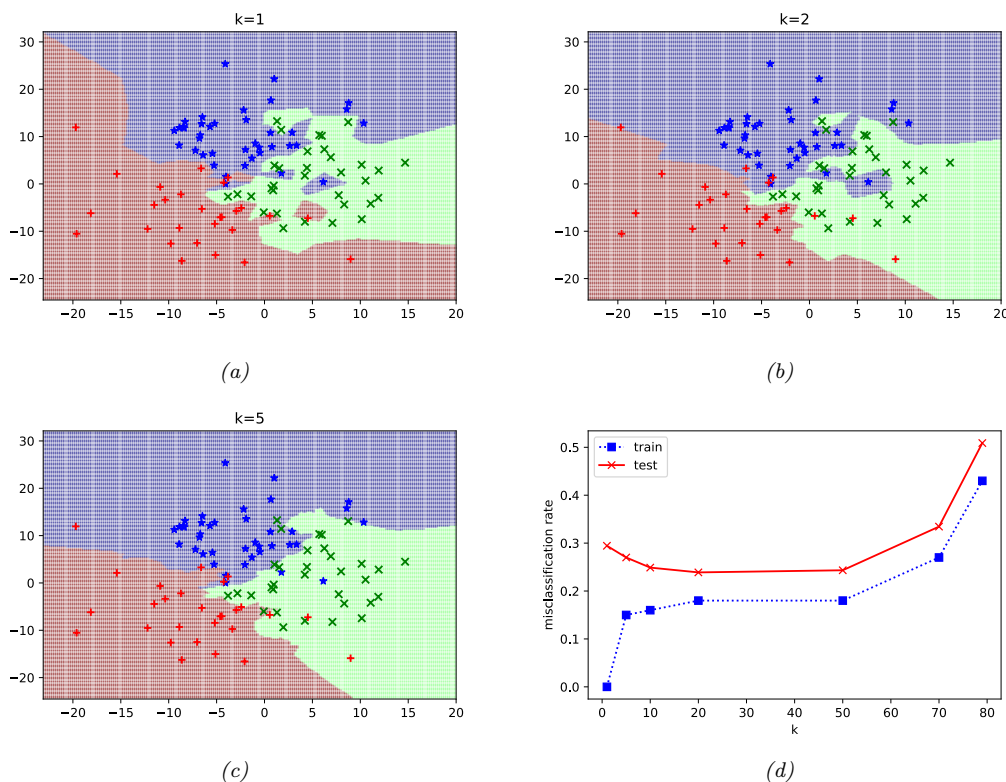


Figure 16.2: Decision boundaries induced by a KNN classifier. (a) $K = 1$. (b) $K = 2$. (c) $K = 5$. (d) Train and test error vs K . Generated by `knn_classify_demo.ipynb`.

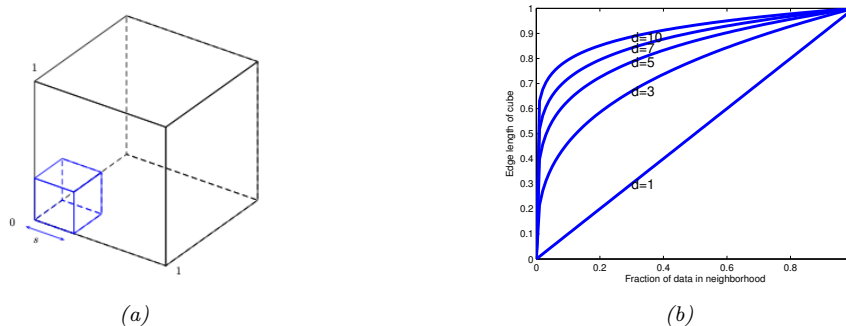


Figure 16.3: Illustration of the curse of dimensionality. (a) We embed a small cube of side s inside a larger unit cube. (b) We plot the edge length of a cube needed to cover a given volume of the unit cube as a function of the number of dimensions. Adapted from Figure 2.6 from [HTF09]. Generated by `curse_dimensionality_plot.ipynb`.

data, we have $e_{10}(0.1) = 0.8$, so we need to extend the cube 80% along each dimension around $\mathbf{x}$. Even if we only use 1% of the data, we find $e_{10}(0.01) = 0.63$. Since the range of the data is only 0 to 1 along each dimension, we see that the method is no longer very local, despite the name “nearest neighbor”. The trouble with looking at neighbors that are so far away is that they may not be good predictors about the behavior of the function at a given point.

There are two main solutions to the curse: make some assumptions about the form of the function (i.e., use a parametric model), and/or use a metric that only cares about a subset of the dimensions (see Section 16.2).

16.1.3 Reducing the speed and memory requirements

KNN classifiers store all the training data. This is obviously very wasteful of space. Various heuristic pruning techniques have been proposed to remove points that do not affect the decision boundaries, see e.g., [WM00]. In Section 17.4, we discuss a more principled approach based on a sparsity promoting prior; the resulting method is called a sparse kernel machine, and only keeps a subset of the most useful exemplars.

In terms of running time, the challenge is to find the K nearest neighbors in less than $O(N)$ time, where N is the size of the training set. Finding exact nearest neighbors is computationally intractable when the dimensionality of the space goes above about 10 dimensions, so most methods focus on finding the approximate nearest neighbors. There are two main classes of techniques, based on partitioning space into regions, or using hashing.

For partitioning methods, one can either use some kind of **k-d tree**, which divides space into axis-parallel regions, or some kind of clustering method, which uses anchor points. For hashing methods, **locality sensitive hashing (LSH)** [GIM99] is widely used, although more recent methods learn the hashing function from data (see e.g., [Wan+15]). See [LRU14] for a good introduction to hashing methods.

An open-source library called **FAISS**, for efficient exact and approximate nearest neighbor search (and K-means clustering) of dense vectors, is available at <https://github.com/facebookresearch/faiss>, and described in [JDJ17].

16.1.4 Open set recognition

Ask not what this is called, ask what this is like. — Moshe Bar.[Bar09]

In all of the classification problems we have considered so far, we have assumed that the set of classes $\mathcal{C}$ is fixed. (This is an example of the **closed world assumption**, which assumes there is a fixed number of (types of) things.) However, many real world problems involve test samples that come from new categories. This is called **open set recognition**, as we discuss below.

16.1.4.1 Online learning, OOD detection and open set recognition

For example, suppose we train a face recognition system to predict the identity of a person from a fixed set or **gallery** of face images. Let $\mathcal{D}_t = \{(\mathbf{x}_n, y_n) : \mathbf{x}_n \in \mathcal{X}, y_n \in \mathcal{C}_t, n = 1 : N_t\}$ be the labeled dataset at time t , where $\mathcal{X}$ is the set of (face) images, and $\mathcal{C}_t = \{1, \dots, C_t\}$ is the set of people known to the system at time t (where $C_t \leq t$). At test time, the system may encounter a new person that it has not seen before. Let $\mathbf{x}_{t+1}$ be this new image, and $y_{t+1} = C_{t+1}$ be its new label. The system

needs to recognize that the input is from a new category, and not accidentally classify it with a label from $\mathcal{C}_t$. This is called **novelty detection**. In this case, the input is being generated from the distribution $p(\mathbf{x}|y = C_{t+1})$, where $C_{t+1} \notin \mathcal{C}_t$ is the new “class label”. Detecting that $\mathbf{x}_{t+1}$ is from a novel class may be hard if the appearance of this new image is similar to the appearance of any of the existing images in $\mathcal{D}_t$.

If the system is successful at detecting that $\mathbf{x}_{t+1}$ is novel, then it may ask for the id of this new instance, call it C_{t+1} . It can then add the labeled pair $(\mathbf{x}_{t+1}, C_{t+1})$ to the dataset to create $\mathcal{D}_{t+1}$, and can grow the set of unique classes by adding C_{t+1} to $\mathcal{C}_t$ (c.f., [JK13]). This is called **incremental learning**, **online learning**, **life-long learning**, or **continual learning**. At future time points, the system may encounter an image sampled from $p(\mathbf{x}|y = c)$, where c is an existing class, or where c is a new class, or the image may be sampled from some entirely different kind of distribution $p'(\mathbf{x})$ unrelated to faces (e.g., someone uploads a photo of their dog). (Detecting this latter kind of event is called **out-of-distribution** or **OOD** detection.)

In this online setting, we often only get a few (sometimes just one) example of each class. Prediction in this setting is known as **few-shot classification**, and is discussed in more detail in Section 19.6. KNN classifiers are well-suited to this task. For example, we can just store all the instances of each class in a gallery of examples, as we explained above. At time $t + 1$, when we get input $\mathbf{x}_{t+1}$, rather than predicting a label for $\mathbf{x}_{t+1}$ by comparing it to some parametric model for each class, we just find the example in the gallery that is nearest (most similar) to $\mathbf{x}_{t+1}$, call it $\mathbf{x}'$. We then need to determine if $\mathbf{x}'$ and $\mathbf{x}_{t+1}$ are sufficiently similar to constitute a match. (In the context of person classification, this is known as **person re-identification** or **face verification**, see e.g., [WSH16]). If there is no match, we can declare the input to be novel or OOD.

The key ingredient for all of the above problems is the (dis)similarity metric between inputs. We discuss ways to learn this in Section 16.2.

16.1.4.2 Other open world problems

The problem of open-set recognition, and incremental learning, are just examples of problems that require the **open world assumption** c.f., [Rus15]. There are many other examples of such problems.

For example, consider the problem of **entity resolution**, called **entity linking**. In this problem, we need to determine if different strings (e.g., “John Smith” and “Jon Smith”) refer to the same entity or not. See e.g. [SHF15] for details.

Another important application is in **multi-object tracking**. For example, when a radar system detects a new “blip”, is it due to an existing missile that is being tracked, or is it a new objective that has entered the airspace? An elegant mathematical framework for dealing with such problems, known as **random finite sets**, is described in [Mah07; Mah13; Vo+15].

16.2 Learning distance metrics

Being able to compute the “semantic distance” between a pair of points, $d(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^+$ for $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$, or equivalently their similarity $s(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^+$, is of crucial importance to tasks such as nearest neighbor classification (Section 16.1), self-supervised learning (Section 19.2.4.4), similarity-based clustering (Section 21.5), content-based retrieval, visual tracking, etc.

When the input space is $\mathcal{X} = \mathbb{R}^D$, the most common distance metric is the Mahalanobis distance

$$d_{\mathbf{M}}(\mathbf{x}, \mathbf{x}') = \sqrt{(\mathbf{x} - \mathbf{x}')^{\top} \mathbf{M} (\mathbf{x} - \mathbf{x}')} \quad (16.3)$$

We discuss some methods to learn the matrix $\mathbf{M}$ in Section 16.2.1. For high dimensional inputs, or structured inputs, it is better to first learn an embedding $\mathbf{e} = f(\mathbf{x})$, and then to compute distances in embedding space. When f is a DNN, this is called **deep metric learning**; we discuss this in Section 16.2.2.

16.2.1 Linear and convex methods

In this section, we discuss some methods that try to learn the Mahalanobis distance matrix $\mathbf{M}$, either directly (as a convex problem), or indirectly via a linear projection. For other approaches to metric learning, see e.g., [Kul13; Kim19] for more details.

16.2.1.1 Large margin nearest neighbors

In [WS09], they propose to learn the Mahalanobis matrix $\mathbf{M}$ so that the resulting distance metric works well when used by a nearest neighbor classifier. The resulting method is called **large margin nearest neighbor** or **LMNN**.

This works as follows. For each example data point i , let N_i be a set of **target neighbors**; these are usually chosen to be the set of K points with the same class label that are closest in Euclidean distance. We now optimize $\mathbf{M}$ so that we minimize the distance between each point i and all of its target neighbors $j \in N_i$:

$$\mathcal{L}_{\text{pull}}(\mathbf{M}) = \sum_{i=1}^N \sum_{j \in N_i} d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_j)^2 \quad (16.4)$$

We also want to ensure that examples with incorrect labels are far away. To do this, we ensure that each example i is closer (by some margin $m \geq 0$) to its target neighbors j than to other points l with different labels (so-called **impostors**). We can do this by minimizing

$$\mathcal{L}_{\text{push}}(\mathbf{M}) = \sum_{i=1}^N \sum_{j \in N_i} \sum_{l=1}^N \mathbb{I}(y_i \neq y_l) [m + d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_j)^2 - d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_l)^2]_+ \quad (16.5)$$

where $[z]_+ = \max(z, 0)$ is the hinge loss function (Section 4.3.2). The overall objective is $\mathcal{L}(\mathbf{M}) = (1 - \lambda)\mathcal{L}_{\text{pull}}(\mathbf{M}) + \lambda\mathcal{L}_{\text{push}}(\mathbf{M})$, where $0 < \lambda < 1$. This is a convex function defined over a convex set, which can be minimized using **semidefinite programming**. Alternatively, we can parameterize the problem using $\mathbf{M} = \mathbf{W}^{\top} \mathbf{W}$, and then minimize wrt $\mathbf{W}$ using unconstrained gradient methods. This is no longer convex, but allows us to use a low-dimensional mapping $\mathbf{W}$.

For large datasets, we need to tackle the $O(N^3)$ cost of computing Equation (16.5). We discuss some speedup tricks in Section 16.2.5.

16.2.1.2 Neighborhood components analysis

Another way to learn a linear mapping $\mathbf{W}$ such that $\mathbf{M} = \mathbf{W}^{\top} \mathbf{W}$ is known as **neighborhood components analysis** or **NCA** [Gol+05]. This defines the probability that sample $\mathbf{x}_i$ has $\mathbf{x}_j$ as its

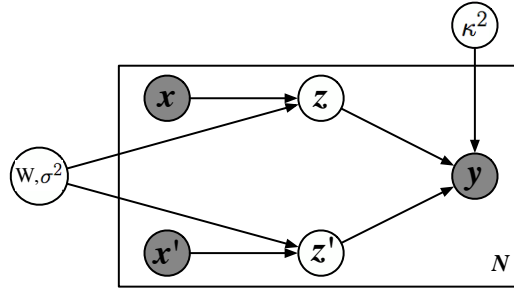


Figure 16.4: Illustration of latent coincidence analysis (LCA) as a directed graphical model. The inputs $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^D$ are mapped into Gaussian latent variables $\mathbf{z}, \mathbf{z}' \in \mathbb{R}^L$ via a linear mapping $\mathbf{W}$. If the two latent points coincide (within length scale κ) then we set the similarity label to $y = 1$, otherwise we set it to $y = 0$. From Figure 1 of [DS12]. Used with kind permission of Lawrence Saul.

nearest neighbor using the linear softmax function

$$p_{ij}^{\mathbf{W}} = \frac{\exp(-\|\mathbf{W}\mathbf{x}_i - \mathbf{W}\mathbf{x}_j\|_2^2)}{\sum_{l \neq i} \exp(-\|\mathbf{W}\mathbf{x}_i - \mathbf{W}\mathbf{x}_l\|_2^2)} \quad (16.6)$$

(This is a supervised version of stochastic neighborhood embeddings discussed in Section 20.4.10.1.) The expected number of correctly classified examples according to a 1NN classifier using distance $\mathbf{W}$ is given by $J(\mathbf{W}) = \sum_{i=1}^N \sum_{j \neq i: y_j = y_i} p_{ij}^{\mathbf{W}}$. Let $\mathcal{L}(\mathbf{W}) = 1 - J(\mathbf{W})/N$ be the leave one out error. We can minimize $\mathcal{L}$ wrt $\mathbf{W}$ using gradient methods.

16.2.1.3 Latent coincidence analysis

Yet another way to learn a linear mapping $\mathbf{W}$ such that $\mathbf{M} = \mathbf{W}^T \mathbf{W}$ is known as **latent coincidence analysis** or **LCA** [DS12]. This defines a conditional latent variable model for mapping a pair of inputs, $\mathbf{x}$ and $\mathbf{x}'$, to a label $y \in \{0, 1\}$, which specifies if the inputs are similar (e.g., have same class label) or dissimilar. Each input $\mathbf{x} \in \mathbb{R}^D$ is mapped to a low dimensional latent point $\mathbf{z} \in \mathbb{R}^L$ using a stochastic mapping $p(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}|\mathbf{W}\mathbf{x}, \sigma^2 \mathbf{I})$, and $p(\mathbf{z}'|\mathbf{x}') = \mathcal{N}(\mathbf{z}'|\mathbf{W}\mathbf{x}', \sigma^2 \mathbf{I})$. (Compare this to factor analysis, discussed in Section 20.2.) We then define the probability that the two inputs are similar using $p(y = 1|\mathbf{z}, \mathbf{z}') = \exp(-\frac{1}{2\kappa^2} \|\mathbf{z} - \mathbf{z}'\|)$. See Figure 16.4 for an illustration of the modeling assumptions.

We can maximize the log marginal likelihood $\ell(\mathbf{W}, \sigma^2, \kappa^2) = \sum_n \log p(y_n|\mathbf{x}_n, \mathbf{x}'_n)$ using the EM algorithm (Section 8.7.2). (We can set $\kappa = 1$ WLOG, since it just changes the scale of $\mathbf{W}$.) More precisely, in the E step, we compute the posterior $p(\mathbf{z}, \mathbf{z}'|\mathbf{x}, \mathbf{x}', y)$ (which can be done in closed form), and in the M step, we solve a weighted least squares problem (c.f., Section 13.6.2). EM will monotonically increase the objective, and does not need step size adjustment, unlike the gradient based methods used in NCA (Section 16.2.1.2). (It is also possible to use variational Bayes (Section 4.6.8.3) to fit this model, as well as various sparse and nonlinear extensions, as discussed in [ZMY19].)

16.2.2 Deep metric learning

When measuring the distance between high-dimensional or structured inputs, it is very useful to first learn an embedding to a lower dimensional “semantic” space, where distances are more meaningful, and less subject to the curse of dimensionality (Section 16.1.2). Let $\mathbf{e} = f(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}^L$ be an embedding of the input that preserves the “relevant” semantic aspects of the input, and let $\hat{\mathbf{e}} = \mathbf{e}/\|\mathbf{e}\|_2$ be the ℓ_2 -normalized version. This ensures that all points lie on a hyper-sphere. We can then measure the distance between two points using the normalized Euclidean distance

$$d(\mathbf{x}_i, \mathbf{x}_j; \boldsymbol{\theta}) = \|\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j\|_2^2 \quad (16.7)$$

where smaller values means more similar, or the cosine similarity

$$d(\mathbf{x}_i, \mathbf{x}_j; \boldsymbol{\theta}) = \hat{\mathbf{e}}_i^\top \hat{\mathbf{e}}_j \quad (16.8)$$

where larger values means more similar. (Cosine similarity measures the angle between the two vectors, as illustrated in Figure 20.43.) These quantities are related via

$$\|\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j\|_2^2 = (\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j)^\top (\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j) = 2 - 2\hat{\mathbf{e}}_i^\top \hat{\mathbf{e}}_j \quad (16.9)$$

This overall approach is called **deep metric learning** or DML.

The basic idea in DML is to learn the embedding function such that similar examples are closer than dissimilar examples. More precisely, we assume we have a labeled dataset, $\mathcal{D} = \{(\mathbf{x}_i, y_i) : i = 1 : N\}$, from which we can derive a set of similar pairs, $\mathcal{S} = \{(i, j) : y_i = y_j\}$. If $(i, j) \in \mathcal{S}$ but $(i, k) \notin \mathcal{S}$, then we assume that $\mathbf{x}_i$ and $\mathbf{x}_j$ should be close in embedding space, whereas $\mathbf{x}_i$ and $\mathbf{x}_k$ should be far. We discuss various ways to enforce this property below. Note that these methods also work when we do not have class labels, provided we have some other way of defining similar pairs. For example, in Section 19.2.4.3, we discuss self-supervised approaches to representation learning, that automatically create semantically similar pairs, and learn embeddings to force these pairs to be closer than unrelated pairs.

Before discussing DML in more detail, it is worth mentioning that many recent approaches to DML are not as good as they claim to be, as pointed out in [MBL20; Rot+20]. (The claims in some of these papers are often invalid due to improper experimental comparisons, a common flaw in contemporary ML research, as discussed in e.g., [BLV19; LS19b].) We therefore focus on (slightly) older and simpler methods, that tend to be more robust.

16.2.3 Classification losses

Suppose we have labeled data with C classes. Then we can fit a classification model in $O(NC)$ time, and then reuse the hidden features as an embedding function. (It is common to use the second-to-last layer, since it generalizes better to new classes than the final layer.) This approach is simple and scalable. However, it only learns to embed examples on the correct side of a decision boundary, which does not necessarily result in similar examples being placed close together and dissimilar examples being placed far apart. In addition, this method cannot be used if we do not have labeled training data.

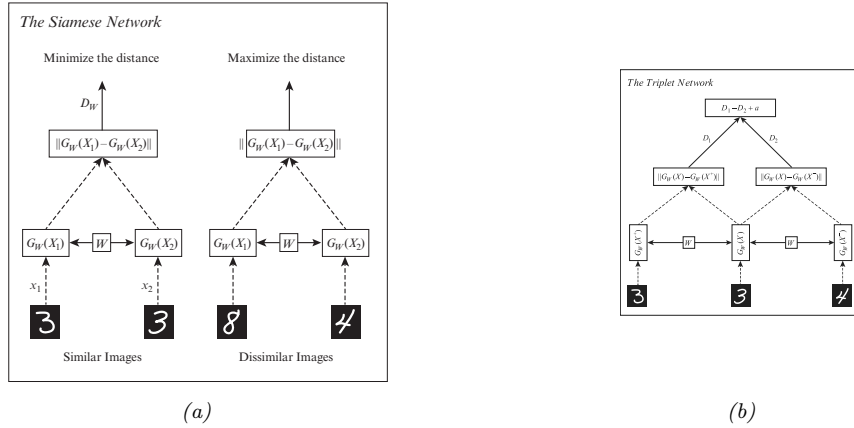


Figure 16.5: Networks for deep metric learning. (a) Siamese network. (b) Triplet network. Adapted from Figure 5 of [KB19].

16.2.4 Ranking losses

In this section, we consider minimizing **ranking loss**, to ensure that similar examples are closer than dissimilar examples. Most of these methods do not need class labels (although we sometimes assume that labels exist as a notationally simple way to define similarity).

16.2.4.1 Pairwise (contrastive) loss and Siamese networks

One of the earliest approaches to representation learning from similar/dissimilar pairs was based on minimizing the following **contrastive loss** [CHL05]:

$$\mathcal{L}(\theta; \mathbf{x}_i, \mathbf{x}_j) = \mathbb{I}(y_i = y_j) d(\mathbf{x}_i, \mathbf{x}_j)^2 + \mathbb{I}(y_i \neq y_j) [m - d(\mathbf{x}_i, \mathbf{x}_j)]_+^2 \quad (16.10)$$

where $[z]_+ = \max(0, z)$ is the hinge loss and $m > 0$ is a margin parameter. Intuitively, we want to force positive pairs (with the same label) to be close, and negative pairs (with different labels) to be further apart than some minimal safety margin. We minimize this loss over all pairs of data. Naively this takes $O(N^2)$ time; see Section 16.2.5 for some speedups.

Note that we use the same feature extractor $f(\cdot; \theta)$ for both inputs, $\mathbf{x}_i$ and $\mathbf{x}_j$. when computing the distance, as illustrated in Figure 16.5a. The resulting network is therefore called a **Siamese network** (named after Siamese twins).

16.2.4.2 Triplet loss

One disadvantage of pairwise losses is that the optimization of the positive pairs is independent of the negative pairs, which can make their magnitudes incomparable. A solution to this is to use the **triplet loss** [SKP15]. This is defined as follows. For each example i (known as an **anchor**), we find a similar (positive) example $\mathbf{x}_i^+$ and a dissimilar (negative) example $\mathbf{x}_i^-$. We then minimize the following loss, averaged overall all triples:

$$\mathcal{L}(\theta; \mathbf{x}_i, \mathbf{x}_i^+, \mathbf{x}_i^-) = [d_\theta(\mathbf{x}_i, \mathbf{x}_i^+)^2 - d_\theta(\mathbf{x}_i, \mathbf{x}_i^-)^2 + m]_+ \quad (16.11)$$

Intuitively this says we want the distance from the anchor to the positive to be less (by some safety margin m) than the distance from the anchor to the negative. We can compute the triplet loss using a triplet network as shown in Figure 16.5b.

Naively minimizing triplet loss takes $O(N^3)$ time. In practice we compute the loss on a minibatch (chosen so that there is at least one similar and one dissimilar example for the anchor point, often taken to be the first entry in the minibatch). Nevertheless the method can be slow. We discuss some speedups in Section 16.2.5.

16.2.4.3 N-pairs loss

One problem with the triplet loss is that each anchor is only compared to one negative example at a time. This might not provide a strong enough learning signal. One solution to this is to create a multi-class classification problem in which we create a set of $N - 1$ negatives and 1 positive for every anchor. This is called the **N-pairs loss** [Soh16]. More precisely, we define the following loss for each set:

$$\mathcal{L}(\theta; \mathbf{x}, \mathbf{x}^+, \{\mathbf{x}_k^-\}_{k=1}^{N-1}) = \log \left(1 + \sum_{k=1}^{N-1} \exp [\hat{\mathbf{e}}_{\theta}(\mathbf{x})^{\top} \hat{\mathbf{e}}_{\theta}(\mathbf{x}_k^-) - \hat{\mathbf{e}}_{\theta}(\mathbf{x})^{\top} \hat{\mathbf{e}}_{\theta}(\mathbf{x}^+)] \right) \quad (16.12)$$

$$= -\log \frac{\exp(\hat{\mathbf{e}}_{\theta}(\mathbf{x})^{\top} \hat{\mathbf{e}}_{\theta}(\mathbf{x}^+))}{\exp(\hat{\mathbf{e}}_{\theta}(\mathbf{x})^{\top} \hat{\mathbf{e}}_{\theta}(\mathbf{x}^+)) + \sum_{k=1}^{N-1} \exp(\hat{\mathbf{e}}_{\theta}(\mathbf{x})^{\top} \hat{\mathbf{e}}_{\theta}(\mathbf{x}_k^-))} \quad (16.13)$$

Note that the N-pairs loss is the same as the **InfoNCE** loss used in the CPC paper [OLV18]. In [Che+20a], they propose a version where they scale the similarities by a temperature term; they call this the **NT-Xent** (normalized temperature-scaled cross-entropy) loss. We can view the temperature parameter as scaling the radius of the hypersphere on which the data lives.

When $N = 2$, the loss reduces to the logistic loss

$$\mathcal{L}(\theta; \mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \log (1 + \exp(\hat{\mathbf{e}}_{\theta}(\mathbf{x})^{\top} \hat{\mathbf{e}}_{\theta}(\mathbf{x}^-) - \hat{\mathbf{e}}_{\theta}(\mathbf{x})^{\top} \hat{\mathbf{e}}_{\theta}(\mathbf{x}^+))) \quad (16.14)$$

Compare this to the margin loss used by triplet learning (when $m = 1$):

$$\mathcal{L}(\theta; \mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \max (0, \hat{\mathbf{e}}(\mathbf{x})^{\top} \hat{\mathbf{e}}(\mathbf{x}^-) - \hat{\mathbf{e}}(\mathbf{x})^{\top} \hat{\mathbf{e}}(\mathbf{x}^+) + 1) \quad (16.15)$$

See Figure 4.2 for a comparison of these two functions.

16.2.5 Speeding up ranking loss optimization

The main disadvantage of ranking loss is the $O(N^2)$ or $O(N^3)$ cost of computing the loss function, due to the need to compare all pairs or triples of examples. In this section, we discuss various speedup tricks.

16.2.5.1 Mining techniques

A key insight is that we don't need to consider all negative examples for each anchor, since most will be uninformative (i.e., will incur zero loss). Instead we can focus attention on negative examples which are closer to the anchor than its nearest positive example. These are called **hard negatives**, and are particularly useful for speeding up triplet loss.

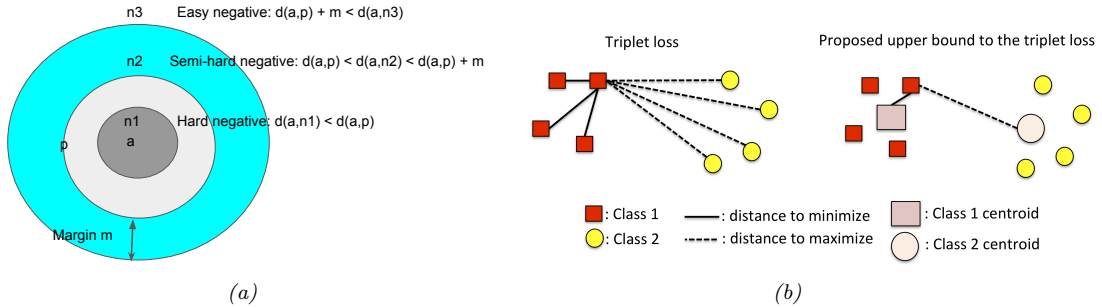


Figure 16.6: Speeding up triplet loss minimization. (a) Illustration of hard vs easy negatives. Here a is the anchor point, p is a positive point, and n_i are negative points. Adapted from Figure 4 of [KB19]. (b) Standard triplet loss would take $8 \times 3 \times 4 = 96$ calculations, whereas using a proxy loss (with one proxy per class) takes $8 \times 2 = 16$ calculations. From Figure 1 of [Do+19]. Used with kind permission of Gustavo Cerneiro.

More precisely, if a is an anchor and p is its nearest positive example, we say that n is a hard negative (for a) if $d(\mathbf{x}_a, \mathbf{x}_n) < d(\mathbf{x}_a, \mathbf{x}_p)$ and $y_n \neq y_a$. Sometimes an anchor may not have any hard negatives. We can therefore increase the pool of candidates by considering **semi-hard negatives**, for which

$$d(\mathbf{x}_a, \mathbf{x}_p) < d(\mathbf{x}_a, \mathbf{x}_n) < d(\mathbf{x}_a, \mathbf{x}_p) + m \quad (16.16)$$

where $m > 0$ is a margin parameter. See Figure 16.6a for an illustration. This is the technique used by Google’s **FaceNet** model [SKP15], which learns an embedding function for faces, so it can cluster similar looking faces together, to which the user can attach a name.

In practice, the hard negatives are usually chosen from within the minibatch. This therefore requires large batch sizes to ensure sufficient diversity. Alternatively, we can have a separate process that continually updates the set of candidate hard negatives, as the distance measure evolves during training.

16.2.5.2 Proxy methods

Triplet loss minimization is expensive even with hard negative mining (Section 16.2.5.1). Ideally we can find a method that is $O(N)$ time, just like classification loss.

One such method, proposed in [MA+17], measures the distance between each anchor and a set of P **proxies** that represent each class, rather than directly measuring distance between examples. These proxies need to be updated online as the distance metric evolves during learning. The overall procedure takes $O(NP^2)$ time, where $P \sim C$.

More recently, [Qia+19] proposed to represent each class with multiple prototypes, while still achieving linear time complexity, using a **soft triplet** loss.

16.2.5.3 Optimizing an upper bound

[Do+19] proposed a simple and fast method for optimizing the triplet loss. The key idea is to define one *fixed* proxy or centroid per class, and then to use distance to the proxy as an upper bound on

the triplet loss.

More precisely, consider a simplified form of the triplet loss, without the margin term:

$$\ell_t(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) = \|\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j\| - \|\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_k\| \quad (16.17)$$

where $\hat{\mathbf{e}}_i = \hat{\mathbf{e}}_\theta(\mathbf{x}_i)$, etc. Using the triangle inequality we have

$$\|\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_j\| \leq \|\hat{\mathbf{e}}_i - \mathbf{c}_{y_i}\| + \|\hat{\mathbf{e}}_j - \mathbf{c}_{y_i}\| \quad (16.18)$$

$$\|\hat{\mathbf{e}}_i - \hat{\mathbf{e}}_k\| \geq \|\hat{\mathbf{e}}_i - \mathbf{c}_{y_k}\| - \|\hat{\mathbf{e}}_k - \mathbf{c}_{y_k}\| \quad (16.19)$$

Hence

$$\ell_t(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \leq \ell_u(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \triangleq \|\hat{\mathbf{e}}_i - \mathbf{c}_{y_i}\| - \|\hat{\mathbf{e}}_i - \mathbf{c}_{y_k}\| + \|\hat{\mathbf{e}}_j - \mathbf{c}_{y_i}\| + \|\hat{\mathbf{e}}_k - \mathbf{c}_{y_k}\| \quad (16.20)$$

We can use this to derive a tractable upper bound on the triplet loss as follows:

$$\mathcal{L}_t(\mathcal{D}, \mathcal{S}) = \sum_{(i,j) \in \mathcal{S}, (i,k) \notin \mathcal{S}, i,j,k \in \{1, \dots, N\}} \ell_t(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \leq \sum_{(i,j) \in \mathcal{S}, (i,k) \notin \mathcal{S}, i,j,k \in \{1, \dots, N\}} \ell_u(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k) \quad (16.21)$$

$$= C' \sum_{i=1}^N \left(\|\mathbf{x}_i - \mathbf{c}_{y_i}\| - \frac{1}{3(C-1)} \sum_{m=1, m \neq y_i}^C \|\mathbf{x}_i - \mathbf{c}_m\| \right) \triangleq \mathcal{L}_u(\mathcal{D}, \mathcal{S}) \quad (16.22)$$

where $C' = 3(C-1)(\frac{N}{C} - 1)\frac{N}{C}$ is a constant, and we assume that $\sum_{i=1}^N \mathbb{I}(y_i = c) = N/C$ for each c . It is clear that $\mathcal{L}_u$ can be computed in $O(NC)$ time. See Figure 16.6b for an illustration.

In [Do+19], they show that $0 \leq \mathcal{L}_t - \mathcal{L}_u \leq \frac{N^3}{C^2} K$, where K is some constant that depends on the spread of the centroids. To ensure the bound is tight, the centroids should be as far from each other as possible, and the distances between them should be as similar as possible. An easy way to ensure is to define the $\mathbf{c}_m$ vectors to be one-hot vectors, one per class. These vectors already have unit norm, and are orthogonal to each other. The distance between each pair of centroids is $\sqrt{2}$, which ensures the upper bound is fairly tight.

The downside of this approach is that it assumes the embedding layer is $L = C$ dimensional. There are two solutions to this. First, after training, we can add a linear projection layer to map from C to $L \neq C$, or we can take the second-to-last layer of the embedding network. The second approach is to sample a large number of points on the L -dimensional unit hyper-sphere (which we can do by sampling from the standard normal, and then normalizing [Mar72]), and then running K-means clustering (Section 21.3) with $K = C$. In the experiments reported in [Do+19], these two approaches give similar results.

Interestingly, in [Rot+20], they show that increasing $\pi_{\text{intra}}/\pi_{\text{inter}}$ results in improved downstream performance on various retrieval tasks, where

$$\pi_{\text{intra}} = \frac{1}{Z_{\text{intra}}} \sum_{c=1}^C \sum_{i \neq j: y_i = y_j = c} d(\mathbf{x}_i, \mathbf{x}_j) \quad (16.23)$$

is the average intra-class distance, and

$$\pi_{\text{inter}} = \frac{1}{Z_{\text{inter}}} \sum_{c=1}^C \sum_{c'=1}^C d(\boldsymbol{\mu}_c, \boldsymbol{\mu}_{c'}) \quad (16.24)$$

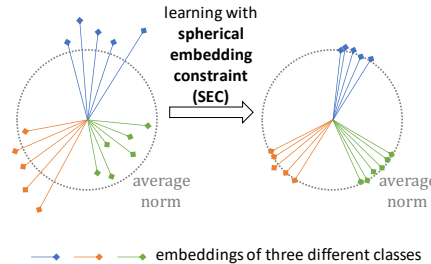


Figure 16.7: Adding spherical embedding constraint to a deep metric learning method. Used with kind permission of Dingyi Zhang.

is the average inter-class distance, where $\mu_c = \frac{1}{Z_c} \sum_{i:y_i=c} \hat{e}_i$ is the mean embedding for examples from class c . This suggests that we should not only keep the centroids far apart (in order to maximize the numerator), but we should also prevent examples from getting too close to their centroids (in order to minimize the denominator); this latter term is not captured in the method of [Do+19].

16.2.6 Other training tricks for DML

Besides the speedup tricks in Section 16.2.5, there are a lot of other details that are important to get right in order to ensure good DML performance. Many of these details are discussed in [MBL20; Rot+20]. Here we just briefly mention a few.

One important issue is how the minibatches are created. In classification problems (at least with balanced classes), selecting examples at random from the training set is usually sufficient. However, for DML, we need to ensure that each example has some other examples in the minibatch that are similar to it, as well as some others that are dissimilar to it. One approach is to use hard mining techniques (Section 16.2.5.1). Another idea is to use coreset methods applied to previously learned embeddings to select a diverse minibatch at each step [Sin+20]. However, [Rot+20] show that the following simple strategy also works well for creating each batch: pick B/n classes, and then pick N_c examples randomly from each class, where B is the batch size, and $N_c = 2$ is a tuning parameter.

Another important issue is avoiding overfitting. Since most datasets used in the DML literature are small, it is standard to use an image classifier, such as GoogLeNet (Section 14.3.3) or ResNet (Section 14.3.4), which has been pre-trained on ImageNet, and then to fine-tune the model using the DML loss. (See Section 19.2 for more details on this kind of transfer learning.) In addition, it is standard to use data augmentation (see Section 19.1). (Indeed, with some self-supervised learning methods, data aug is the only way to create similar pairs.)

In [ZLZ20], they propose to add a **spherical embedding constraint (SEC)**, which is an additional batchwise regularization term, which encourages all the examples to have the same norm. That is, the regularizer is just the empirical variance of the norms of the (unnormalized) embeddings in that batch. See Figure 16.7 for an illustration. This regularizer can be added to any of the existing DML losses to modestly improve training speed and stability, as well as final performance, analogously to how batchnorm (Section 14.2.4.1) is used.

16.3 Kernel density estimation (KDE)

In this section, we consider a form of non-parametric density estimation known as **kernel density estimation** or **KDE**. This is a form of generative model, since it defines a probability distribution $p(\mathbf{x})$ that can be evaluated pointwise, and which can be sampled from to generate new data.

16.3.1 Density kernels

Before explaining KDE, we must define what we mean by a “kernel”. This term has several different meanings in machine learning and statistics.¹ In this section, we use a specific kind of kernel which we refer to as a **density kernel**. This is a function $\mathcal{K} : \mathbb{R} \rightarrow \mathbb{R}_+$ such that $\int \mathcal{K}(x)dx = 1$ and $\mathcal{K}(-x) = \mathcal{K}(x)$. This latter symmetry property implies the $\int x\mathcal{K}(x)dx = 0$, and hence

$$\int x\mathcal{K}(x - x_n)dx = x_n \quad (16.25)$$

A simple example of such a kernel is the **boxcar kernel**, which is the uniform distribution within the unit interval around the origin:

$$\mathcal{K}(x) \triangleq 0.5\mathbb{I}(|x| \leq 1) \quad (16.26)$$

Another example is the **Gaussian kernel**:

$$\mathcal{K}(x) = \frac{1}{(2\pi)^{\frac{1}{2}}} e^{-x^2/2} \quad (16.27)$$

We can control the width of the kernel by introducing a **bandwidth** parameter h :

$$\mathcal{K}_h(x) \triangleq \frac{1}{h} \mathcal{K}\left(\frac{x}{h}\right) \quad (16.28)$$

We can generalize to vector valued inputs by defining a **radial basis function** or **RBF** kernel:

$$\mathcal{K}_h(\mathbf{x}) \propto \mathcal{K}_h(\|\mathbf{x}\|) \quad (16.29)$$

In the case of the Gaussian kernel, this becomes

$$\mathcal{K}_h(\mathbf{x}) = \frac{1}{h^D (2\pi)^{D/2}} \prod_{d=1}^D \exp\left(-\frac{1}{2h^2} x_d^2\right) \quad (16.30)$$

Although Gaussian kernels are popular, they have unbounded support. Some alternative kernels, which have compact support (which can be computationally faster), are listed in Table 16.1. See Figure 16.8 for a plot of these kernel functions.

1. For a good blog post on this, see <https://francisbach.com/cursed-kernels/>.

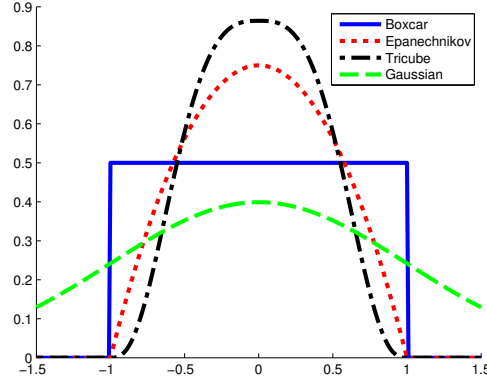


Figure 16.8: A comparison of some popular normalized kernels. Generated by [smoothingKernelPlot.ipynb](#).

Name	Definition	Compact	Smooth	Boundaries
Gaussian	$\mathcal{K}(x) = (2\pi)^{-\frac{1}{2}} e^{-x^2/2}$	0	1	1
Boxcar	$\mathcal{K}(x) = \frac{1}{2} \mathbb{I}(x \leq 1)$	1	0	0
Epanechnikov kernel	$\mathcal{K}(x) = \frac{3}{4} (1 - x^2) \mathbb{I}(x \leq 1)$	1	1	0
Tri-cube kernel	$\mathcal{K}(x) = \frac{70}{81} (1 - x ^3)^3 \mathbb{I}(x \leq 1)$	1	1	1

Table 16.1: List of some popular normalized kernels in 1d. Compact=1 means the function is non-zero for a finite range of inputs. Smooth=1 means the function is differentiable over the range of its support. Boundaries=1 means the function is also differentiable at the boundaries of its support.

16.3.2 Parzen window density estimator

To explain how to use kernels to define a nonparametric density estimate, recall the form of the Gaussian mixture model from Section 3.5.1. If we assume a fixed spherical Gaussian covariance and uniform mixture weights, we get

$$p(\mathbf{x}|\boldsymbol{\theta}) = \frac{1}{K} \sum_{k=1}^K \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \sigma^2 \mathbf{I}) \quad (16.31)$$

One problem with this model is that it requires specifying the number K of clusters, as well as their locations $\boldsymbol{\mu}_k$. An alternative to estimating these parameters is to allocate one cluster center per data point. In this case, the model becomes

$$p(\mathbf{x}|\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \mathcal{N}(\mathbf{x}|\mathbf{x}_n, \sigma^2 \mathbf{I}) \quad (16.32)$$

We can generalize Equation (16.32) by writing

$$p(\mathbf{x}|\mathcal{D}) = \frac{1}{N} \sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \quad (16.33)$$

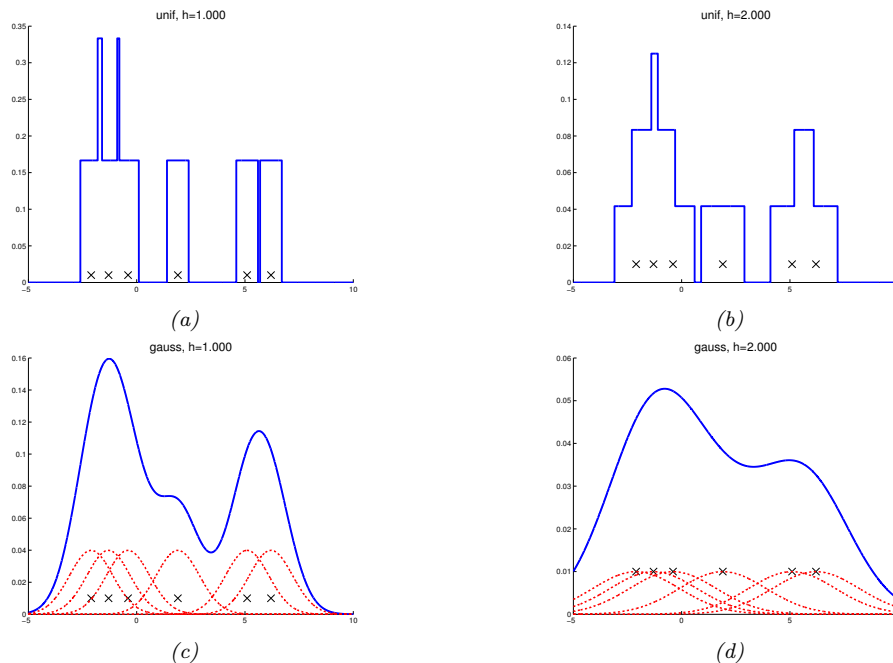


Figure 16.9: A nonparametric (Parzen) density estimator in 1d estimated from 6 data points, denoted by x . Top row: uniform kernel. Bottom row: Gaussian kernel. Left column: bandwidth parameter $h = 1$. Right column: bandwidth parameter $h = 2$. Adapted from http://en.wikipedia.org/wiki/Kernel_density_estimation. Generated by `parzen_window_demo2.ipynb`.

where $\mathcal{K}_h$ is a density kernel. This is called a **Parzen window density estimator**, or **kernel density estimator (KDE)**.

The advantage over a parametric model is that no model fitting is required (except for choosing h , discussed in Section 16.3.3), and there is no need to pick the number of cluster centers. The disadvantage is that the model takes a lot of memory (you need to store all the data) and a lot of time to evaluate.

Figure 16.9 illustrates KDE in 1d for two kinds of kernel. On the top, we use a boxcar kernel; the resulting model just counts how many data points land within an interval of size h around each x_n to get a piecewise constant density. On the bottom, we use a Gaussian kernel, which results in a smoother density.

16.3.3 How to choose the bandwidth parameter

We see from Figure 16.9 that the bandwidth parameter h has a large effect on the learned distribution. We can view this as controlling the complexity of the model.

In the case of 1d data, where the “true” data generating distribution is assumed to be a Gaussian, one can show [BA97a] that the optimal bandwidth for a Gaussian kernel (from the point of view of

minimizing frequentist risk) is given by $h = \sigma \left(\frac{4}{3N} \right)^{1/5}$. We can compute a robust approximation to the standard deviation by first computing the **median absolute deviation**, $\text{median}(|\mathbf{x} - \text{median}(\mathbf{x})|)$, and then using $\hat{\sigma} = 1.4826 \text{ MAD}$. If we have D dimensions, we can estimate h_d separately for each dimension, and then set $h = (\prod_{d=1}^D h_d)^{1/D}$.

16.3.4 From KDE to KNN classification

In Section 16.1, we discussed the K nearest neighbor classifier as a heuristic approach to classification. Interestingly, we can derive it as a generative classifier in which the class conditional densities $p(\mathbf{x}|y = c)$ are modeled using KDE. Rather than using a fixed bandwidth and counting how many data points fall within the hyper-cube centered on a datapoint, we will allow the bandwidth or volume to be different for each data point. Specifically, we will “grow” a volume around $\mathbf{x}$ until we encounter K data points, regardless of their class label. This is called a **balloon kernel density estimator** [TS92]. Let the resulting volume have size $V(\mathbf{x})$ (this was previously h^D), and let there be $N_c(\mathbf{x})$ examples from class c in this volume. Then we can estimate the class conditional density as follows:

$$p(\mathbf{x}|y = c, \mathcal{D}) = \frac{N_c(\mathbf{x})}{N_c V(\mathbf{x})} \quad (16.34)$$

where N_c is the total number of examples in class c in the whole data set. If we take the class prior to be $p(y = c) = N_c/N$, then the class posterior is given by

$$p(y = c|\mathbf{x}, \mathcal{D}) = \frac{\frac{N_c(\mathbf{x})}{N_c V(\mathbf{x})} \frac{N_c}{N}}{\sum_{c'} \frac{N_{c'}(\mathbf{x})}{N_{c'} V(\mathbf{x})} \frac{N_{c'}}{N}} = \frac{N_c(\mathbf{x})}{\sum_{c'} N_{c'}(\mathbf{x})} = \frac{N_c(\mathbf{x})}{K} = \frac{1}{K} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n = c) \quad (16.35)$$

where we used the fact that $\sum_c N_c(\mathbf{x}) = K$, since we choose a total of K points (regardless of class) around every point. This matches Equation (16.1).

16.3.5 Kernel regression

Just as KDE can be used for generative classifiers (see Section 16.1), it can also be used for generative models for regression, as we discuss below.

16.3.5.1 Nadaraya-Watson estimator for the mean

In regression, our goal is to compute the conditional expectation

$$\mathbb{E}[y|\mathbf{x}, \mathcal{D}] = \int y p(y|\mathbf{x}, \mathcal{D}) dy = \frac{\int y p(\mathbf{x}, y|\mathcal{D}) dy}{\int p(\mathbf{x}, y|\mathcal{D}) dy} \quad (16.36)$$

If we use an MVN for $p(y, \mathbf{x}|\mathcal{D})$, we derive a result which is equivalent to linear regression, as we showed in Section 11.2.3.5. However, the assumption that $p(y, \mathbf{x}|\mathcal{D})$ is Gaussian is rather limiting. We can use KDE to more accurately approximate the joint density $p(\mathbf{x}, y|\mathcal{D})$ as follows:

$$p(y, \mathbf{x}|\mathcal{D}) \approx \frac{1}{N} \sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \mathcal{K}_h(y - y_n) \quad (16.37)$$

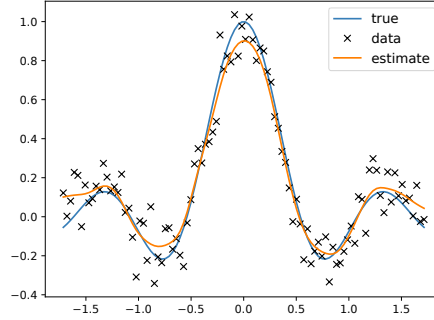


Figure 16.10: An example of kernel regression in 1d using a Gaussian kernel. Generated by [kernelRegression-Demo.ipynb](#).

Hence

$$\mathbb{E}[y|\mathbf{x}, \mathcal{D}] = \frac{\frac{1}{N} \sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \int y \mathcal{K}_h(y - y_n) dy}{\frac{1}{N} \sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'}) \int \mathcal{K}_h(y - y_{n'}) dy} \quad (16.38)$$

We can simplify the numerator using the fact that $\int y \mathcal{K}_h(y - y_n) dy = y_n$ (from Equation (16.25)). We can simplify the denominator using the fact that density kernels integrate to one, i.e., $\int \mathcal{K}_h(y - y_n) dy = 1$. Thus

$$\mathbb{E}[y|\mathbf{x}, \mathcal{D}] = \frac{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) y_n}{\sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'})} = \sum_{n=1}^N y_n w_n(\mathbf{x}) \quad (16.39)$$

$$w_n(\mathbf{x}) \triangleq \frac{\mathcal{K}_h(\mathbf{x} - \mathbf{x}_n)}{\sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'})} \quad (16.40)$$

We see that the prediction is just a weighted sum of the outputs at the training points, where the weights depend on how similar $\mathbf{x}$ is to the stored training points. This method is called **kernel regression**, **kernel smoothing**, or the **Nadaraya-Watson** (N-W) model. See Figure 16.10 for an example, where we use a Gaussian kernel.

In Section 17.2.3, we discuss the connection between kernel regression and Gaussian process regression.

16.3.5.2 Estimator for the variance

Sometimes it is useful to compute the predictive variance, as well as the predictive mean. We can do this by noting that

$$\mathbb{V}[y|\mathbf{x}, \mathcal{D}] = \mathbb{E}[y^2|\mathbf{x}, \mathcal{D}] - \mu(\mathbf{x})^2 \quad (16.41)$$

where $\mu(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}, \mathcal{D}]$ is the N-W estimate. If we use a Gaussian kernel with variance σ^2 , we can compute $\mathbb{E}[y^2|\mathbf{x}, \mathcal{D}]$ as follows:

$$\mathbb{E}[y^2|\mathbf{x}, \mathcal{D}] = \frac{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \int y^2 \mathcal{K}_h(y - y_n) dy}{\sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'}) \int \mathcal{K}_h(y - y_{n'}) dy} \quad (16.42)$$

$$= \frac{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) (\sigma^2 + y_n^2)}{\sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'})} \quad (16.43)$$

where we used the fact that

$$\int y^2 \mathcal{N}(y|y_n, \sigma^2) dy = \sigma^2 + y_n^2 \quad (16.44)$$

Combining Equation (16.43) with Equation (16.41) gives

$$\mathbb{V}[y|\mathbf{x}, \mathcal{D}] = \sigma^2 + \sum_{n=1}^N w_n(\mathbf{x}) y_n^2 - \mu(\mathbf{x})^2 \quad (16.45)$$

This matches Eqn. 8 of [BA10] (modulo the initial σ^2 term).

16.3.5.3 Locally weighted regression

We can drop the normalization term from Equation (16.39) to get

$$\mu(\mathbf{x}) = \sum_{n=1}^N y_n \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \quad (16.46)$$

This is just a weighted sum of the observed responses, where the weights depend on how similar the test input $\mathbf{x}$ is to the training points $\mathbf{x}_n$.

Rather than just interpolating the stored responses y_n , we can fit a locally linear model around each training point:

$$\mu(\mathbf{x}) = \min_{\beta} \sum_{n=1}^N [y_n - \beta^T \phi(\mathbf{x}_n)]^2 \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) \quad (16.47)$$

where $\phi(\mathbf{x}) = [1, \mathbf{x}]$. This is called **locally linear regression** (LRR) or **locally-weighted scatter-plot smoothing**, and is commonly known by the acronym **LOWESS** or **LOESS** [CD88]. This is often used when annotating scatter plots with local trend lines.